class 5 notes

limited questions in class Mon/Wed; fewer than years past; too easy? confused? spread-out classroom? good time to ask questions about any confusion - if you have question I can almost guarantee others do too or ask in Ed or OHs; good Ed posts (substantive questions or answering other students will get credit for class participation)

st243-sh

Regular expressions are tech for working with strings to detect/match/extract/replace patterns

can get very complicated - I know regex reasonably well, but there's lots of complexity I've never learned as not needed or becasue could do a bit of programming in R/python that avoids need for complicated regex

syntax is:

- complicated (very concise)
- hard to read
- special characters are overloaded (multiple meanings for some of them when used in different ways/contexts)
- chars are used as special chars but also needed as literal chars
- nested processing (first by shell, Python, etc., then by regex engine)
    - chars might be interpreted by the program calling the regex engine; need to get the regex into the regex engine
- different regex versions (basic, extended, Perl-style)
- differences in different languages/context (shell vs. Python)

Some examples:

grep "." vs grep "[.?]" just "." means any character

would need "." so treated as a literal character by the regex processor

grep "[.!?]" but "!" means history in shell (show it) "[.!?]"

! is special in the shell but not in regex . and ? are special chars in regex but not inside []

probably fine in Python

and this is fine in shell '[.!?]'

but note that "[!.?]" seems ok

when have a problem, try escaping a character so it is treated literally by the shell

AI: good, but not perfect tests!!!!

I find it useful to know some regex off the top of my head, but I will go look up/remind for more complicated

Match cat or at or t:

PollEv #1

echo "ct" | grep -E -o "c?a?t"

some other basic test cases that shouldn't be detected: "ca", "c", "a"

Claude cat/at/t session: https://claude.ai/chat/b56f2fd6-c866-474a-a095-94585e939600

what happens without -E

echo "ct" | grep -o "c?a?t"

echo "c?a?t" | grep -o "c?a?t" # ? is not special in basic REs

PollEv #2

cat|at|t includes all of "cat" it's not ca(t|a)(t|t)

echo caat | grep "cat|at|t"

why use (cat|at|t)?

echo sat | grep "s(cat|at|t)"

PollEv #3

echo "cat in the hat" | grep -E -o "^cat|at|t$"

what "cat" and "at"

Enumeration is a good way to start

avoid premature optimization/conciseness

get one you know is correct, then can compare results on tests with a more sophisticated/complicated one

cat|at|t

(ca|a)?t

(c?a)?t

which is more clear?

Let's work on a regex that matches any integer or decimal-valued number, positive or negative

Google form: st243-re

coming up with tests can clarify what you are trying to do

deal with cases where you know all the inputs

deal with cases where you might get weird/mistaken input

deal with cases where you might get adversarial input

on board: correct, incorrect, unsure

-3

75

7.

0.7

.7

5e9

5.3e9

0.5e9

5e-9

2.7e-9

word

5@3

5cf37

-0

00

7,000

surrounded by whitespace?

PollEv #4

Is this correct?

"\-?[0-9] \.*?[0-9]*"

enumerate:

"\-?([0-9]+|[0-9]+\.[0-9]+|\.[0-9]+)"

"\-?(|[0-9]+\.|\.)[0-9]+"

"\-?([0-9]+\.|\.)?[0-9]+"

"\-?(([0-9]+)?\.)?[0-9]+"

(why need "\-"?) because grep is interpreting "-" as part of indicating an option; need to get "-" into the regex processor

I would handle scientific notation by enumeration

now try Claude?

Why is greedy matching the default?

echo "99 red balloons go by" | grep -P -o "[0-9]+?"

? overloaded (not 0 or 1 here)

suppose I have some html and want to extract or replace the html tags:

text="Do an internship in place of one course."

grep -E -o "<.*>"

See bash tutorial for challenge of how to avoid need for greedy matching

"<[^<]+>"

(Google form? st243-swap) Challenge: how change date format (using capturing groups)

Use sed to convert date stamps like "6/26/1985" to "1985-06-26"

sed "s/([0-9]{1,2})\/([0-9]{1,2})\/([0-9]{4})/\3-\2-\1/g"